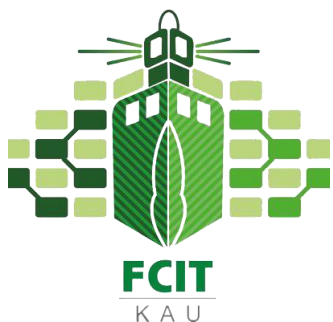




King Abdul-Aziz University
Faculty of Computing and Information Technology
Computer Science Department
Algorithms and Data Structures (II)
CPCS-324 Spring 2023 Group Project – Part 2



EMPIRICAL ANALYSIS OF DIJKSTRA ALGORITHMS

Project Team:

Name	ID	Section
Razan Arif Alamri		
Shatha Kalid Binmahfouz		
Khlood Homiid Alsofyani		
Leen Baqalaql		



Table of Contents:

1. Introduction	4
2. Empirical Analysis	5
3. Difficulties Faced During the Phase Design	9
4. Conclusion	9
5. References.....	9
6. Appendix.....	9



Illustrations:

TABLE 1: RUN TIME OF DIJKSTRA ALGORITHM	7
TABLE 2:EMPIRICAL TIME EFFICIENCY FOR DIJKSTRA ALGORITHM.....	8
TABLE 3:RATIO OF TIME EFFICIENCY FOR DIJKSTRA ALGORITHM	8
FIGURE 1: DIJKSTRA ALGORITHM	4
FIGURE 2: OUTPUT OF SHORTEST PATH WITH ITS LENGTH FOR GRAPH READ FROM FILE	5
FIGURE 3: OUTPUT OF SHORTEST PATH WITH ITS LENGTH FOR GRAPH READ FROM FILE	6
FIGURE 4:OUTPUT OF SHORTEST PATH WITH ITS LENGTH FOR GRAPH READ FROM FILE	6
FIGURE 5:OUTPUT OF SHORTEST PATH WITH ITS LENGTH FOR GRAPH READ FROM FILE	7



1. Introduction

In this project, we are faced with a problem based on Air freight where we need to transport goods to specific locations over several kilometers. We compare an algorithm based on Greedy technique which is the Dijkstra algorithm. In this problem, we want to use an algorithm to find the all-pair-source shortest path problem. Dijkstra method is employed to identify the shortest route between a particular source and every other vertex. This is a special way for Dijkstra to solve problems. The aim is to find the shortest path between any two vertices by computing the minimum spanning tree of a graph.

2. Dijkstra algorithm:

The objective is to solve the single-source shortest-paths problem in a connected graph with weighted edges, where a specific vertex is designated as the source. Dijkstra's algorithm goal is to find the shortest path from the source vertex to all other vertices in the graph. It's important to note that the problem does not require finding a single shortest path that visits all other vertices, which would be much more challenging. Instead, the problem requires a set of paths.

ALGORITHM *Dijkstra(G, s)*

```
//Dijkstra's algorithm for single-source shortest paths
//Input: A weighted connected graph  $G = \langle V, E \rangle$  with nonnegative weights
//      and its vertex  $s$ 
//Output: The length  $d_v$  of a shortest path from  $s$  to  $v$ 
//      and its penultimate vertex  $p_v$  for every vertex  $v$  in  $V$ 
Initialize( $Q$ ) //initialize priority queue to empty
for every vertex  $v$  in  $V$ 
     $d_v \leftarrow \infty$ ;  $p_v \leftarrow \text{null}$ 
    Insert( $Q, v, d_v$ ) //initialize vertex priority in the priority queue
 $d_s \leftarrow 0$ ; Decrease( $Q, s, d_s$ ) //update priority of  $s$  with  $d_s$ 
 $V_T \leftarrow \emptyset$ 
for  $i \leftarrow 0$  to  $|V| - 1$  do
     $u^* \leftarrow \text{DeleteMin}(Q)$  //delete the minimum priority element
     $V_T \leftarrow V_T \cup \{u^*\}$ 
    for every vertex  $u$  in  $V - V_T$  that is adjacent to  $u^*$  do
        if  $d_{u^*} + w(u^*, u) < d_u$ 
             $d_u \leftarrow d_{u^*} + w(u^*, u)$ ;  $p_u \leftarrow u^*$ 
            Decrease( $Q, u, d_u$ )
```

Figure 1: Dijkstra algorithm

3. Empirical Analysis

- **The Problem:**

“Air freight is a system of transporting goods which can be commercial or non-commercial by aircraft. ... Aircraft can travel for a certain number of kilometers to deliver freight from one location to another. ... The problem, however, is an optimization problem to cut down the distances travelled minimizing the number of stops-over so as to optimize the time for delivery of time-sensitive goods and minimize the total cost “.

- **Requirement 1:**

1.1 Implement the Dijkstra algorithm to compute the length of the shortest paths from all locations to the rest of the locations. Make sure to print the shortest paths with their lengths.

```
-----
----- Welcome to Air Freight Application -----
-----

This program to Compute the shortest path between every pair of locations using two algorithms:

1- Dijkstra algorithm computing the shortest path for a specified source
2- Dijkstra-based all shortest path algorithm

-----

<< Test Type >>

1: Read The Graph From File
2: Create a Random Graph

> From these choices, please enter your choice: 1

-----

Shortest paths from all location:

Shortest paths from location A:
Loc.A: city 0 Loc.A: city 0 --- route length: 0
Loc.A: city 0 Loc.A: city 0 Loc.F: city 5 Loc.B: city 1 --- route length: 8
Loc.A: city 0 Loc.A: city 0 Loc.F: city 5 Loc.B: city 1 Loc.C: city 2 --- route length: 11
Loc.A: city 0 Loc.A: city 0 Loc.F: city 5 Loc.B: city 1 Loc.C: city 2 Loc.D: city 3 --- route length: 15
Loc.A: city 0 Loc.A: city 0 Loc.F: city 5 Loc.B: city 1 Loc.E: city 4 --- route length: 11
Loc.A: city 0 Loc.A: city 0 Loc.F: city 5 --- route length: 5
Loc.A: city 0 Loc.A: city 0 Loc.F: city 5 Loc.B: city 1 Loc.E: city 4 Loc.G: city 6 --- route length: 13
Loc.A: city 0 Loc.A: city 0 Loc.F: city 5 Loc.B: city 1 Loc.C: city 2 Loc.H: city 7 --- route length: 16
Loc.A: city 0 Loc.A: city 0 Loc.F: city 5 Loc.B: city 1 Loc.C: city 2 Loc.D: city 3 Loc.I: city 8 --- route length: 19
Loc.A: city 0 Loc.A: city 0 Loc.F: city 5 Loc.J: city 9 --- route length: 7

-----
Shortest paths from location B:
```

Figure 2: Output of shortest path with its length for graph read from file

```

Shortest paths from location B:
Loc.B: city 1 has no path to Loc.A: city 0
Loc.B: city 1 Loc.B: city 1 --- route length: 0
Loc.B: city 1 Loc.B: city 1 Loc.C: city 2 --- route length: 3
Loc.B: city 1 Loc.B: city 1 Loc.C: city 2 Loc.D: city 3 --- route length: 7
Loc.B: city 1 Loc.B: city 1 Loc.E: city 4 --- route length: 3
Loc.B: city 1 has no path to Loc.F: city 5
Loc.B: city 1 Loc.B: city 1 Loc.E: city 4 Loc.G: city 6 --- route length: 5
Loc.B: city 1 Loc.B: city 1 Loc.C: city 2 Loc.H: city 7 --- route length: 8
Loc.B: city 1 Loc.B: city 1 Loc.C: city 2 Loc.D: city 3 Loc.I: city 8 --- route length: 11
Loc.B: city 1 has no path to Loc.J: city 9
-----
Shortest paths from location C:
Loc.C: city 2 has no path to Loc.A: city 0
Loc.C: city 2 has no path to Loc.B: city 1
Loc.C: city 2 Loc.C: city 2 --- route length: 0
Loc.C: city 2 Loc.C: city 2 Loc.D: city 3 --- route length: 4
Loc.C: city 2 has no path to Loc.E: city 4
Loc.C: city 2 has no path to Loc.F: city 5
Loc.C: city 2 has no path to Loc.G: city 6
Loc.C: city 2 Loc.C: city 2 Loc.H: city 7 --- route length: 5
Loc.C: city 2 Loc.C: city 2 Loc.D: city 3 Loc.I: city 8 --- route length: 8
Loc.C: city 2 has no path to Loc.J: city 9
-----
Shortest paths from location D:
Loc.D: city 3 has no path to Loc.A: city 0
Loc.D: city 3 has no path to Loc.B: city 1
Loc.D: city 3 has no path to Loc.C: city 2
Loc.D: city 3 Loc.D: city 3 --- route length: 0
Loc.D: city 3 has no path to Loc.E: city 4
Loc.D: city 3 has no path to Loc.F: city 5
Loc.D: city 3 has no path to Loc.G: city 6
Loc.D: city 3 has no path to Loc.H: city 7
Loc.D: city 3 Loc.D: city 3 Loc.I: city 8 --- route length: 4
Loc.D: city 3 has no path to Loc.J: city 9
-----

```

Figure 3: Output of shortest path with its length for graph read from file

```

Shortest paths from location E:
Loc.E: city 4 has no path to Loc.A: city 0
Loc.E: city 4 has no path to Loc.B: city 1
Loc.E: city 4 Loc.E: city 4 Loc.C: city 2 --- route length: 4
Loc.E: city 4 Loc.E: city 4 Loc.C: city 2 Loc.D: city 3 --- route length: 8
Loc.E: city 4 Loc.E: city 4 --- route length: 0
Loc.E: city 4 has no path to Loc.F: city 5
Loc.E: city 4 Loc.E: city 4 Loc.G: city 6 --- route length: 2
Loc.E: city 4 Loc.E: city 4 Loc.C: city 2 Loc.H: city 7 --- route length: 9
Loc.E: city 4 Loc.E: city 4 Loc.C: city 2 Loc.D: city 3 Loc.I: city 8 --- route length: 12
Loc.E: city 4 has no path to Loc.J: city 9
-----
Shortest paths from location F:
Loc.F: city 5 has no path to Loc.A: city 0
Loc.F: city 5 Loc.F: city 5 Loc.B: city 1 --- route length: 3
Loc.F: city 5 Loc.F: city 5 Loc.B: city 1 Loc.C: city 2 --- route length: 6
Loc.F: city 5 Loc.F: city 5 Loc.B: city 1 Loc.C: city 2 Loc.D: city 3 --- route length: 10
Loc.F: city 5 Loc.F: city 5 Loc.B: city 1 Loc.E: city 4 --- route length: 6
Loc.F: city 5 Loc.F: city 5 --- route length: 0
Loc.F: city 5 Loc.F: city 5 Loc.B: city 1 Loc.E: city 4 Loc.G: city 6 --- route length: 8
Loc.F: city 5 Loc.F: city 5 Loc.B: city 1 Loc.C: city 2 Loc.H: city 7 --- route length: 11
Loc.F: city 5 Loc.F: city 5 Loc.B: city 1 Loc.C: city 2 Loc.D: city 3 Loc.I: city 8 --- route length: 14
Loc.F: city 5 Loc.F: city 5 Loc.J: city 9 --- route length: 2
-----
Shortest paths from location G:
Loc.G: city 6 has no path to Loc.A: city 0
Loc.G: city 6 has no path to Loc.B: city 1
Loc.G: city 6 has no path to Loc.C: city 2
Loc.G: city 6 Loc.G: city 6 Loc.D: city 3 --- route length: 7
Loc.G: city 6 has no path to Loc.E: city 4
Loc.G: city 6 has no path to Loc.F: city 5
Loc.G: city 6 Loc.G: city 6 --- route length: 0
Loc.G: city 6 has no path to Loc.H: city 7
Loc.G: city 6 Loc.G: city 6 Loc.D: city 3 Loc.I: city 8 --- route length: 11
Loc.G: city 6 has no path to Loc.J: city 9
-----

```

Figure 4: Output of shortest path with its length for graph read from file

```

-----
Shortest paths from location H:
Loc.H: city 7 has no path to Loc.A: city 0
Loc.H: city 7 has no path to Loc.B: city 1
Loc.H: city 7 has no path to Loc.C: city 2
Loc.H: city 7 Loc.H: city 7 Loc.D: city 3 --- route length: 4
Loc.H: city 7 has no path to Loc.E: city 4
Loc.H: city 7 has no path to Loc.F: city 5
Loc.H: city 7 has no path to Loc.G: city 6
Loc.H: city 7 Loc.H: city 7 --- route length: 0
Loc.H: city 7 Loc.H: city 7 Loc.I: city 8 --- route length: 3
Loc.H: city 7 has no path to Loc.J: city 9
-----
Shortest paths from location I:
Loc.I: city 8 has no path to Loc.A: city 0
Loc.I: city 8 has no path to Loc.B: city 1
Loc.I: city 8 has no path to Loc.D: city 3
Loc.I: city 8 has no path to Loc.E: city 4
Loc.I: city 8 has no path to Loc.F: city 5
Loc.I: city 8 has no path to Loc.G: city 6
Loc.I: city 8 has no path to Loc.H: city 7
Loc.I: city 8 Loc.I: city 8 --- route length: 0
Loc.I: city 8 has no path to Loc.J: city 9
-----
Shortest paths from location J:
Loc.J: city 9 has no path to Loc.A: city 0
Loc.J: city 9 Loc.J: city 9 Loc.B: city 1 --- route length: 6
Loc.J: city 9 Loc.J: city 9 Loc.B: city 1 Loc.C: city 2 --- route length: 9
Loc.J: city 9 Loc.J: city 9 Loc.B: city 1 Loc.C: city 2 Loc.D: city 3 --- route length: 13
Loc.J: city 9 Loc.J: city 9 Loc.B: city 1 Loc.E: city 4 --- route length: 9
Loc.J: city 9 has no path to Loc.F: city 5
Loc.J: city 9 Loc.J: city 9 Loc.G: city 6 --- route length: 8
Loc.J: city 9 Loc.J: city 9 Loc.B: city 1 Loc.C: city 2 Loc.H: city 7 --- route length: 14
Loc.J: city 9 Loc.J: city 9 Loc.B: city 1 Loc.C: city 2 Loc.D: city 3 Loc.I: city 8 --- route length: 17
Loc.J: city 9 Loc.J: city 9 --- route length: 0
-----

```

Figure 5: Output of shortest path with its length for graph read from file

- Requirement 2:

2.1 Use the Dijkstra algorithm implemented in requirement 2 to compute the length of the shortest paths for each pair of vertices and apply it on the above five randomly generated graphs. Compute the run duration and compare the algorithm's time efficiency computed from the experiment's results with the one determined by the formula stated in the textbook.

n	m	1	2	3	4	5	Total	Average	Best	Worst
2000	10000	2121	1713	1833	1728	1871	9266	1853.2	1713.0 ms.	2121.0 ms.
3000	15000	4844	6383	5053	6400	5060	27740	5548	4844.0 ms.	6400.0 ms.
4000	20000	10167	10708	10643	10766	10650	52934	10586.8	10167.0 ms.	10766.0 ms.
5000	25000	25354	19563	31022	26892	22620	125451	25090.2	19563.0 ms.	31022.0 ms.
6000	30000	37521	34273	44323	42697	51405	210219	42043.8	34273.0 ms.	51405.0 ms.

Table 1: Run time of Dijkstra algorithm

- Empirical time efficiency and the ratio

We will use the running time to empirically determine the time efficiency of the algorithms and compare it with the theoretical time efficiency.

Dijkstra algorithm finds the shortest path from the source node in time $O(n^2)$. We find the running time and expected time efficiency ratios of Dijkstra algorithm is close

n	Running time of Algorithm	Expected theoretical time efficiency
2000	1853.2	4000000
3000	5548	9000000
4000	10586.8	16000000
5000	25090.2	25000000
6000	42043.8	36000000

Table 2: Empirical time efficiency for Dijkstra Algorithm

n	Ratio of running time	Ratio of Expected time efficiency
2000	2.993741	2.25
3000	1.908219	1.777778
4000	2.369951	1.5625
5000	1.675706	1.44
6000	NA	NA

Table 3: Ratio of time efficiency for Dijkstra Algorithm

The time efficiency ratios for Dijkstra's algorithm, both in terms of empirical and expected theoretical time, are distinct from each other. It is important to note that the empirical time efficiency should be considered as the worse option, as the results tend to exceed the theoretical time. However, when calculating time efficiency, certain factors such as the operating system and background processes on the running device must be taken into account. Although the results obtained may not be very similar in value, both approaches are still considered to be accurate and suitable for such studies.

4. Difficulties Faced During the Phase Design

We faced lots of difficulties in this project which are:

1. How to implement Dijkstra algorithm in all single source.
2. How to understand the details of algorithms.
3. How to find a suitable time with the team to discuss.

5. Conclusion

In conclusion, the empirical analysis project for Dijkstra's algorithm for computing the all-pair source shortest path problem was conducted to compare the performance of this approach in terms of runtime. The experiments were carried out on various sizes of randomly generated graphs, and the results showed that the all-pair source shortest path problem using Dijkstra's algorithm performs an effective method for that purpose.

Overall, the results of this empirical analysis project suggest that the all-pair source shortest path problem using Dijkstra's algorithm can be a good choice for solving shortest path problems in terms of runtime, especially for large graphs.

6. References

- (1) Levitin, A. (2011). Introduction to the Design and Analysis of Algorithms. Pearson Higher Ed

7. Appendix

- **GitHub Link:**

<https://github.com/Razan-Alamri/CPCS324-Projecr-Part2>